

Student Name	farjana .s
Register Number	TU-6253210211012
Project Title	Multi-Echelon Supply Chain Network Design using Mixed-Integer Linear Programming (MILP)
Week	Week 3

WEEK-3 – Business Analytics

Task Name

Multi-Echelon Supply Chain Network Design using Mixed-Integer Linear Programming (MILP)

1. Introduction

A supply chain involves moving products from factories to warehouses and finally to customers. When there are multiple factories, warehouses, and customer locations, it can become difficult to decide the best way to distribute products while keeping costs low.

In this task, we use **Mixed-Integer Linear Programming (MILP)** to design an efficient supply chain network. The main goal is to find the best transportation and warehouse decisions while satisfying factory capacity, warehouse capacity, customer demand, and service-level requirements.

The supply chain is represented as:

Factories → Warehouses → Customer Zones

2. Objective

The main objective of this project is to **minimize the total operating cost of the supply chain**.

The model considers:

- Transportation cost from factories to warehouses
- Transportation cost from warehouses to customers
- Fixed cost of opening warehouses
- Factory production capacity
- Warehouse storage capacity
- Customer demand
- Minimum customer service level

By considering all these factors, the model helps identify the most economical supply chain configuration.

3. Supply Chain Network

For our example, we consider:

Factories

- Factory 1 (F1)
- Factory 2 (F2)
- Factory 3 (F3)

Warehouses

- Warehouse 1 (W1)
- Warehouse 2 (W2)

Customer Zones

- Customer 1 (C1)
- Customer 2 (C2)
- Customer 3 (C3)

The products first move from factories to warehouses and then from warehouses to customers.

4. Decision Variables

The model uses three main decision variables.

Factory to Warehouse

[
 x_{ij}
]

represents the quantity of products transported from factory (i) to warehouse (j).

Warehouse to Customer

[
 y_{jk}
]

represents the quantity of products transported from warehouse (j) to customer (k).

Warehouse Opening

[
 z_j
]

is a binary variable.

- ($z_j = 1$) means the warehouse is opened.
- ($z_j = 0$) means the warehouse is not opened.

The binary variable is what makes this a **Mixed-Integer Linear Programming** problem.

5. Input Data

Factory Capacity

Factory	Maximum Capacity
---------	------------------

F1	500 units
----	-----------

F2	700 units
----	-----------

F3	600 units
----	-----------

Warehouse Capacity

Warehouse	Capacity	Fixed Cost
W1	800 units	₹5,000
W2	1,000 units	₹6,000

Customer Demand

Customer	Demand	Minimum Service Level
C1	400 units	90%
C2	500 units	90%
C3	600 units	90%

The total customer demand is:

[
 $400+500+600=1500\text{ units}$
]

The total factory capacity is:

[
 $500+700+600=1800\text{ units}$
]

Therefore, the available factory capacity is enough to meet the total customer demand.

6. Transportation Costs

Factory to Warehouse

W1 W2

F1 ₹10 ₹15

F2 ₹12 ₹9

F3 ₹8 ₹11

Warehouse to Customer

C1 C2 C3

W1 ₹7 ₹6 ₹10

W2 ₹9 ₹8 ₹5

These values represent the transportation cost for moving one unit of product.

7. Objective Function

The model tries to minimize three types of costs:

1. Factory-to-warehouse transportation cost
2. Warehouse-to-customer transportation cost
3. Warehouse opening cost

The objective function is:

[
Minimize\ Z =
 $\sum_i \sum_j C_{ij} x_{ij}$
+
 $\sum_j \sum_k C_{jk} y_{jk}$
+
]

$$\sum_j F_{jz_j}$$

In simple words, the model tries to find the cheapest possible way to move all products through the supply chain.

8. Constraints

Constraint 1 – Factory Capacity

A factory cannot send more products than it can produce.

$$\left[\sum_{ij} x_{ij} \leq S_i \right]$$

This ensures that the production capacity of every factory is respected.

Constraint 2 – Warehouse Flow Balance

The quantity entering a warehouse should be equal to the quantity leaving it.

$$\left[\sum_{ix} x_{ij} = \sum_{jk} y_{jk} \right]$$

This prevents products from appearing or disappearing inside the warehouse.

Constraint 3 – Warehouse Capacity

A warehouse cannot store or process more products than its capacity.

$$\left[\sum_{jk} y_{jk} \leq H_{jz_j} \right]$$

If the warehouse is closed, ($z_j=0$), so no products can pass through it.

Constraint 4 – Customer Demand

Every customer's required demand must be satisfied.

$$\left[\sum_j y_{jk} = D_k \right]$$

For example, Customer C1 needs 400 units, so the total quantity supplied to C1 must be 400 units.

Constraint 5 – Service Level

The company wants to maintain at least a 90% service level for each customer.

$$\left[\sum_j y_{jk} \geq 0.90 D_k \right]$$

This ensures that the required minimum service level is maintained.

Constraint 6 – Warehouse Decision

The warehouse decision is binary:

$$\left[z_j \in \{0,1\} \right]$$

This means a warehouse can either be opened or closed.

Constraint 7 – Non-Negativity

Transportation quantities cannot be negative.

$$\left[x_{ij}, y_{jk} \geq 0 \right]$$

9. Python Implementation

The MILP model can be implemented in Python using the **PuLP** library.

```
import pulp
```

```
# Sets
```

```
factories = ["F1", "F2", "F3"]
```

```
warehouses = ["W1", "W2"]
```

```
customers = ["C1", "C2", "C3"]
```

```
# Factory capacities
```

```
capacity = {
```

```
    "F1": 500,
```

```
    "F2": 700,
```

```
    "F3": 600
```

```
}
```

```
# Warehouse capacities
```

```
warehouse_capacity = {
```

```
    "W1": 800,
```

```
    "W2": 1000
```

```
}
```

```
# Fixed warehouse costs
```

```
fixed_cost = {
```

```
    "W1": 5000,
```

```
    "W2": 6000
```

```
}
```

```
# Customer demand
```

```
demand = {
```

```
    "C1": 400,
```

```
    "C2": 500,
```

```
    "C3": 600
```

```
}
```

```
# Factory to warehouse transportation costs
```

```
cost_fw = {
```

```
    ("F1", "W1"): 10,
```

```
    ("F1", "W2"): 15,
```

```
    ("F2", "W1"): 12,
```

```
    ("F2", "W2"): 9,
```

```
    ("F3", "W1"): 8,
```

```
    ("F3", "W2"): 11
```

```
}
```

```
# Warehouse to customer transportation costs
```

```
cost_wc = {
```

```
    ("W1", "C1"): 7,
```

```
    ("W1", "C2"): 6,
```

```
    ("W1", "C3"): 10,
```

```
    ("W2", "C1"): 9,
```

```
    ("W2", "C2"): 8,
```

```

        ("W2", "C3"): 5
    }

# Create the optimization model
model = pulp.LpProblem(
    "Supply_Chain_Network_Design",
    pulp.LpMinimize
)

# Decision variables
x = pulp.LpVariable.dicts(
    "Factory_Warehouse",
    [(i, j) for i in factories for j in warehouses],
    lowBound=0
)

y = pulp.LpVariable.dicts(
    "Warehouse_Customer",
    [(j, k) for j in warehouses for k in customers],
    lowBound=0
)

z = pulp.LpVariable.dicts(
    "Warehouse_Open",
    warehouses,
    cat="Binary"

```

```
)
```

```
# Objective function
```

```
model += (
```

```
    pulp.lpSum(
```

```
        cost_fw[i, j] * x[i, j]
```

```
        for i in factories
```

```
        for j in warehouses
```

```
    )
```

```
    +
```

```
    pulp.lpSum(
```

```
        cost_wc[j, k] * y[j, k]
```

```
        for j in warehouses
```

```
        for k in customers
```

```
    )
```

```
    +
```

```
    pulp.lpSum(
```

```
        fixed_cost[j] * z[j]
```

```
        for j in warehouses
```

```
    )
```

```
)
```

```
# Factory capacity constraints
```

```
for i in factories:
```

```
    model += (
```

```
        pulp.lpSum(x[i, j] for j in warehouses)
```

```

        <= capacity[i]
    )

# Warehouse flow balance
for j in warehouses:
    model += (
        pulp.lpSum(x[i, j] for i in factories)
        ==
        pulp.lpSum(y[j, k] for k in customers)
    )

# Warehouse capacity constraints
for j in warehouses:
    model += (
        pulp.lpSum(y[j, k] for k in customers)
        <= warehouse_capacity[j] * z[j]
    )

# Customer demand constraints
for k in customers:
    model += (
        pulp.lpSum(y[j, k] for j in warehouses)
        == demand[k]
    )

# Solve the model

```

```

model.solve(pulp.PULP_CBC_CMD(msg=False))

# Display results

print("Status:", pulp.LpStatus[model.status])

print("Minimum Total Cost:", pulp.value(model.objective))


print("\nFactory to Warehouse:")

for i in factories:

    for j in warehouses:

        if x[i, j].value() > 0:

            print(i, "->", j, ":", x[i, j].value())


print("\nWarehouse to Customer:")

for j in warehouses:

    for k in customers:

        if y[j, k].value() > 0:

            print(j, "->", k, ":", y[j, k].value())


print("\nWarehouse Decisions:")

for j in warehouses:

    print(j, "Open:", z[j].value())

```

10. Expected Result

After running the program, the solver will show whether the model has found an optimal solution.

The output will contain:

- Optimization status
- Minimum total supply chain cost
- Quantity transported from each factory to each warehouse
- Quantity transported from each warehouse to each customer
- Which warehouses should be opened

Example:

Status: Optimal

Minimum Total Cost: XXXXX

Factory to Warehouse:

F1 -> W1 : ...

F2 -> W2 : ...

F3 -> W1 : ...

Warehouse to Customer:

W1 -> C1 : ...

W1 -> C2 : ...

W2 -> C3 : ...

Warehouse Decisions:

W1 Open: 1

W2 Open: 1

The exact values are calculated automatically by the optimization solver.

11. Business Interpretation

The solution helps a company make better supply chain decisions.

Instead of manually deciding where products should be sent, the MILP model finds a cost-effective distribution plan while respecting all the important limitations.

For example, management can use the result to understand:

- Which warehouses should be opened.
 - Which factories should supply each warehouse.
 - How many products should be transported between locations.
 - Whether warehouse capacity is sufficient.
 - Whether customer demand can be satisfied.
 - How transportation costs can be reduced.
-

12. Conclusion

This project demonstrates how **Mixed-Integer Linear Programming** can be used to solve a real-world supply chain network problem.

The model considers factories, warehouses, and customers together and finds an efficient flow of products through the network. By minimizing transportation and warehouse costs while satisfying capacity, demand, and service-level requirements, the company can reduce operating expenses and improve its overall supply chain performance.

Therefore, MILP provides a practical and powerful approach for making **data-driven supply chain network design decisions**.